

JC4004 – Computational Intelligence

Sessions 11 & 12: Genetic algorithms

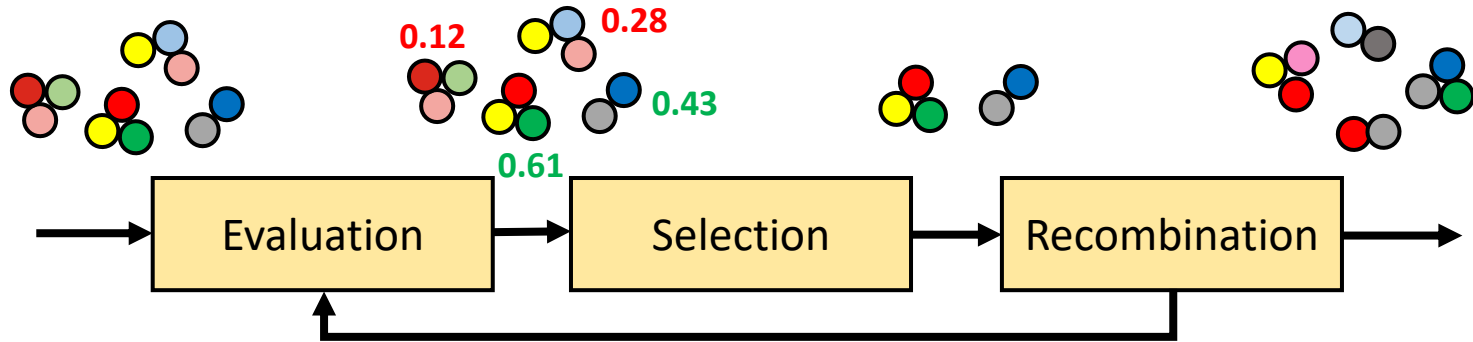
Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)

Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)

Nov-Dec 2024

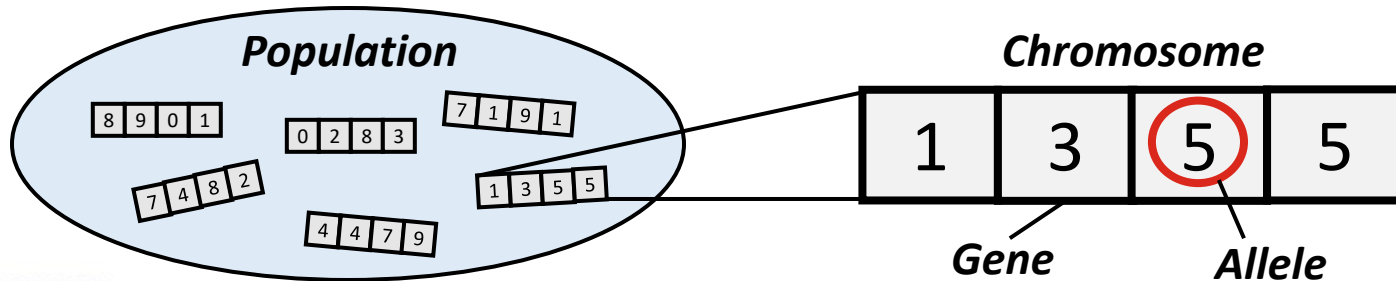
What are genetic algorithms (GA)?

- *Genetic algorithms (GA)* or *evolutionary algorithms* are search algorithms inspired by natural selection
 - Individual fitness for the intended purpose is evaluated
 - The fittest candidates are selected, and the process is repeated
 - Population recombined through crossover and mutation operations



GA terminology

- **Population** is a subset of all the possible encoded solutions to the given problem
- **Chromosome** is one of the possible solutions
- **Gene** is one element position of a chromosome
- **Allele** is the value of a gene for a particular chromosome



Encoding of chromosomes

- Proper encoding of chromosomes is essential to represent the necessary information for the solution
 - Each gene controls a particular characteristic of the individual
- Different encoding methods have been developed to fulfill the requirements of different types of applications
 - Chromosomes usually represented by a vector of numbers, but interpretation of the numbers depends on the encoding
 - Most common encoding methods include *binary encoding*, *value encoding*, and *permutation encoding*

Binary encoding

- In binary encoding, each gene is a bit (0 or 1), representing a particular characteristic of the problem
 - Often 0 represents exclusion and 1 represents inclusion of a member
 - Most common method for encoding

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>Interpretation</i>
Chromosome C_1	1	0	1	1	0	1	$C_1 = \{a, c, d, f\}$
Chromosome C_2	0	1	1	1	0	0	$C_2 = \{b, c, d\}$

Value encoding

- Value encoding uses a specific set of values, such as integers, real numbers, or symbols
 - Efficient for some problems, but often requires application-specific genetic operators to be developed

Integer coding

5	0	12	7	6	6
---	---	----	---	---	---

Real number coding

5.8	-1.1	75.1	975	-16.5	14.2
-----	------	------	-----	-------	------

Symbol coding

↓	←	→	→	↑	←
---	---	---	---	---	---

Permutation encoding

- A set of numbers is used, each number represents an item
 - Useful for ordering problems: the order of numbers in the chromosome represents the order of items

Mapping: $a=1$, $b=2$, $c=3$, $d=4$, $e=5$, $f=6$

*Sequence
represented*

Chromosome C_1

1	3	4	2	6	5
---	---	---	---	---	---

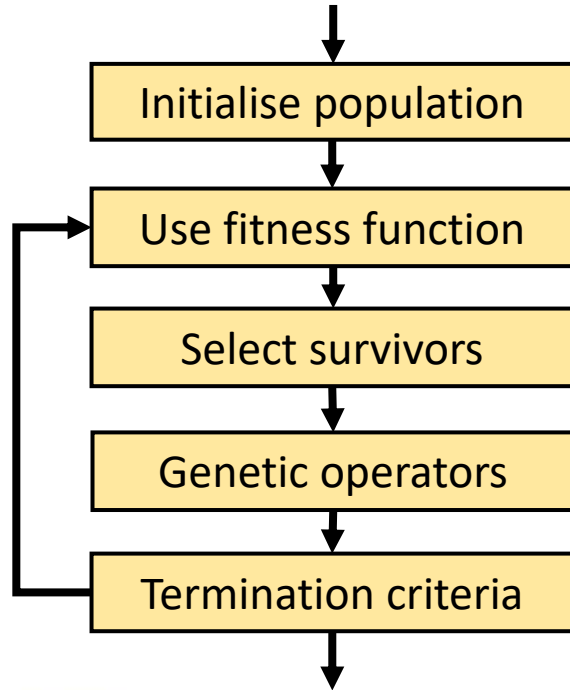
a, c, d, b, f, e

Chromosome C_2

6	5	4	3	2	1
---	---	---	---	---	---

f, e, d, c, b, a

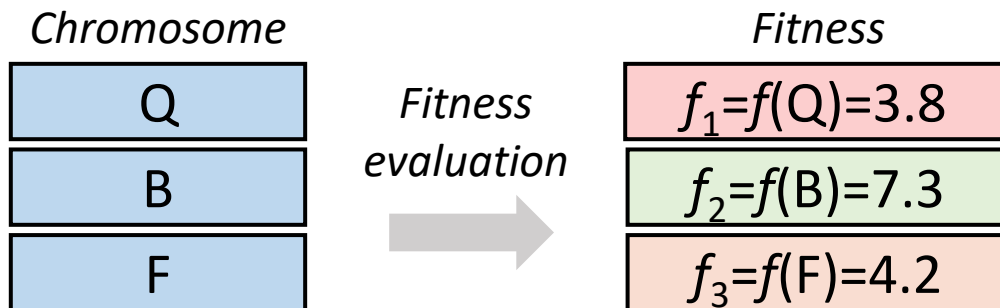
Common genetic algorithm structure



1. Initialise population
 - Random or heuristic initialisation producing hundreds or thousands of chromosomes
2. Apply fitness function to the population
3. Select the fittest chromosomes from the population to form the next generation
4. Generate offsprings using genetic operators
 - Crossover and mutation
5. Test for termination criteria
 - If not met, repeat from step 5, otherwise return the best chromosome

Fitness function

- *Fitness function* takes a candidate solution (chromosome) as input and gives a suitability estimate of the solution as output
 - Definition of the fitness function depends on the problem to be solved
 - The goal is to either maximise or minimise the fitness function, depending on the application (by default, the larger the fitter)



Survivor selection

- Survivors can be selected based on the fitness or the age (i.e., the oldest chromosomes removed first)

Age	Fitness	Chromosome
1	8	A
4	10	C
6	8	Q
4	3	B
3	7	F
3	2	R
2	5	D

Fitness-based
selection



Age	Fitness	Chromosome
5	10	C
2	8	A
7	8	Q

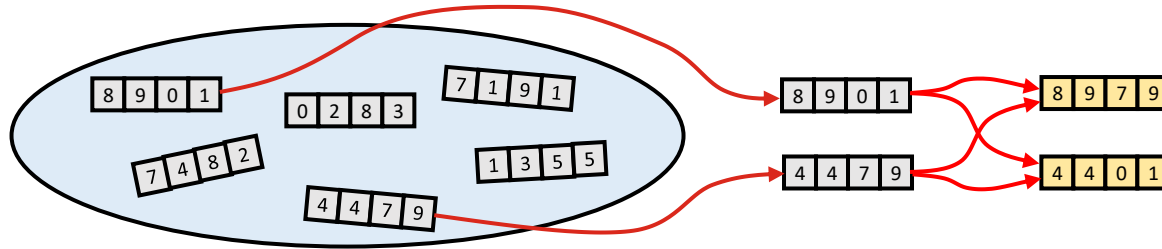
Age-based
selection



Age	Fitness	Chromosome
2	8	A
3	5	D
4	7	F

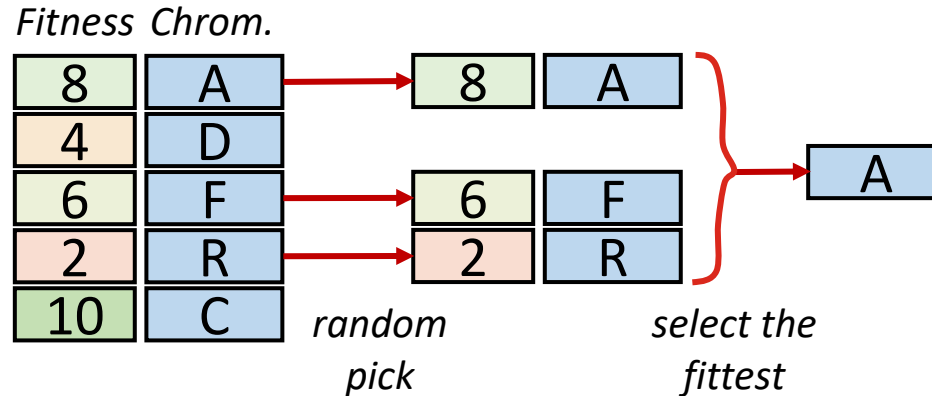
Parent selection

- Subset of survivors selected as parents for offsprings
 - We could simply choose the fittest individuals as parents (i.e. *rank selection*), but this method can converge slower, because the best chromosomes are usually rather similar
- Different selection methods exist to increase genetic variance
 - In this lecture, we introduce *tournament selection* and *roulette-wheel selection*



Tournament selection

- Two phases: first, pick randomly k chromosomes; then, choose the fittest of the picked individuals as parent
- Process is repeated until the desired number of parents are selected

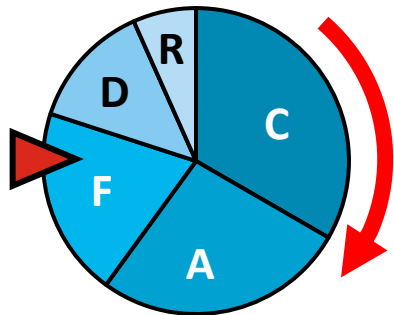


Roulette-wheel selection

- Random selection weighted so that the probability to be selected is proportional to fitness
 - Given fitness value f_i for chromosome i , its probability to be selected from the pool of N chromosomes is $p_i = \frac{f_i}{\sum_{j=1}^N f_j}$

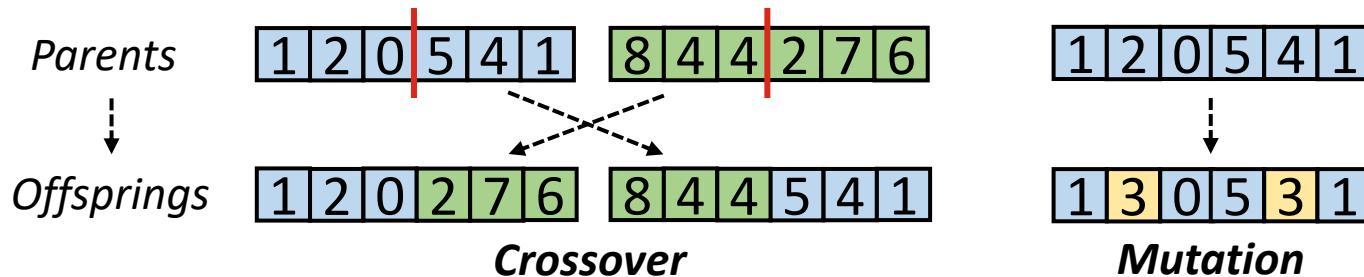
Fitness Chrom.

8	A
4	D
6	F
2	R
10	C



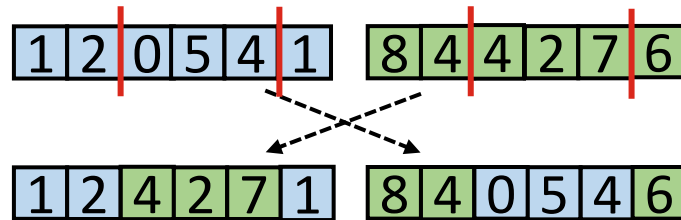
Genetic operators

- The offsprings are generated using *genetic operators*
 - *Crossover*: parts of chromosome swapped between parents
 - *Mutation*: values of some genes (alleles) randomly changed
- Crossover applied first before mutations
 - For smooth convergence, genetic operations should not be too excessive

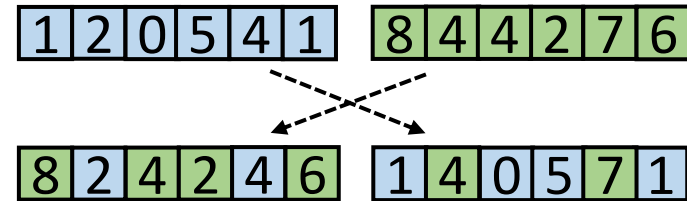


Variants of crossover

- Different variants of crossover exist, some of them shown below
 - *One-point crossover*: crossover applied to two fixed sections
 - *Two- and multi-point crossover*: crossover applied to three (or more) sections in randomly selected positions
 - *Uniform crossover*: for each position, crossover is decided independently with equal probability



Multi-point crossover



Uniform crossover

Variants of mutation

- Some relevant variants of mutations include the following:

1 0 0 1 0 1



1 1 0 1 1 0

Bit flip

1 2 0 5 4 1



1 4 0 5 2 1

Swap

1 2 0 5 4 1



1 5 2 0 4 1

Scramble

1 2 0 5 4 1



1 5 0 2 4 1

Inversion

1 2 0 5 4 1



1 3 0 5 3 1

Small changes

1 2 0 5 4 1



1 0 0 5 0 1

Random resetting

Termination criteria

- Termination criteria may be derived from the fitness function or from a separate *objective function*
- Some possible termination criteria:
 - One of the offsprings provides a full solution to the problem
 - Maximum number of iterations reached
 - The best fitness function outcome has saturated
- After termination, the most fit chromosome of the current population returned as solution

Break: any questions?

Example: solving a Sudoku puzzle

- In Sudoku, you need to have each number 1-4 to appear once in each of the rows, columns, and four 2x2 subgrids
 - The task is to fill in the missing numbers to solve the puzzle

1	a	b	4
c	2	3	d
e	4	1	f
3	g	h	2

- Usually Sudoku uses 9x9 grid, but for simplicity, we use a small 4x4 grid in this example
- For encoding potential solutions, we define a chromosome with 8 genes, each corresponding to one of the missing numbers *a-h*

a	b	c	d	e	f	g	h
---	---	---	---	---	---	---	---

Example: Sudoku fitness function

- Sum of each row and column and 2x2 subgrid should be $1+2+3+4=10$
 - For example, the first row should fulfil equation $1+a+b+4=10 \Rightarrow a+b-5=0$, so the goal is to minimise expression $|a + b - 5|$
 - The full fitness function to minimise can be formulated as:

1	a	b	4
c	2	3	d
e	4	1	f
3	g	h	2

$$f(a, \dots, h) =$$

$$\begin{aligned} &|a + b - 5| + |c + d - 5| + |e + f - 5| + |g + h - 5| \\ &+ |c + e - 6| + |a + g - 4| + |b + h - 6| + |d + f - 4| \\ &+ |a + c - 7| + |b + d - 3| + |e + g - 3| + |f + h - 7| \end{aligned}$$

rows columns

subgrids

Example: Sudoku algorithm

- We can initialise the population by generating 100 vectors of 8 elements, each element with a random value between 1 and 4
 - The mutated genes must also be within interval $[1,4]$: we define a mutation operation $g = [g + \text{rand}(\{-1,1\}) - 1 \bmod 4] + 1$, where g is a gene in a randomly chosen position

1	a	b	4
c	2	3	d
e	4	1	f
3	g	h	2

- At each round, 10 fittest chromosomes are chosen as parents, and 10 offsprings are generated for each
- The algorithm terminates when the fittest chromosome reaches fitness value zero

Sudoku algorithm implementation

```
1 import numpy as np
2 import random
3
4 def fitness(c):
5     res = abs(c[0]+c[1]-5)+abs(c[2]+c[3]-5)+abs(c[4]+c[5]-5)+abs(c[6]+c[7]-5)
6     res += abs(c[2]+c[4]-6)+abs(c[0]+c[6]-4)+abs(c[1]+c[7]-6)+abs(c[3]+c[5]-4)
7     res += abs(c[0]+c[2]-7)+abs(c[1]+c[3]-3)+abs(c[4]+c[6]-3)+abs(c[5]+c[7]-7)
8     return res
9
10 def mutate(c):
11     pos = random.randint(0,len(c)-1)
12     c[pos] = ((c[pos] + random.randint(0,1)*2-1)-1) % 4 + 1
13     return c
14
15 def initialise():
16     population = list()
17     for i in range(100):
18         chromosome = list()
19         for j in range(8):
20             chromosome.append(random.randint(1,4))
21         population.append(chromosome)
22     return population
```

Fitness function

Mutation

Initialisation

Sudoku algorithm implementation

```
24 def run(population):
25     i = 0
26     population.sort(key=fitness)
27     while i < 100 and fitness(population[0]) > 0:
28         new_population = list()
29         for j in range(0,10):
30             parent = population[j]
31             for k in range(0,10):
32                 new_population.append(mutate(parent).copy())
33             population = new_population
34             i += 1
35         population.sort(key=fitness)
36         print(population[0], fitness(population[0]))
37
38 population = initialise()
39 run(population)
```

Termination condition: note the maximum number of iterations!

Main loop

Generate offsprings

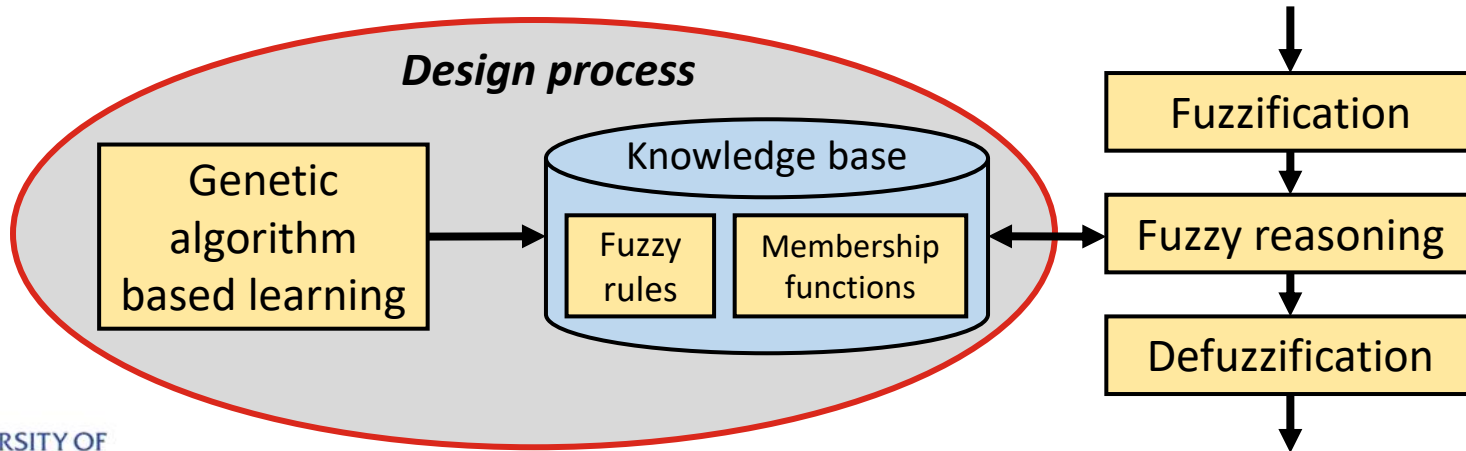
Sudoku algorithm test run

```
24 def run(population):
25     i = 0
26     population.sort(key=fitness)
27     while i < 100 and fitness(population[0]) > 0:
28         new_population = list()
29         for j in range(0,10):
30             parent = population[j]
31             for k in range(0,10):
32                 new_population.append(mutate(parent).copy())
33             population = new_population
34             i += 1
35             population.sort(key=fitness)
36             print(population[0],fitness(population[0]))
37
38 population = initialise()
39 run(population)
```

```
[3, 1, 2, 4, 2, 3, 1, 4] 12
[2, 1, 3, 2, 2, 3, 2, 4] 9
[4, 2, 3, 1, 3, 2, 1, 4] 6
[2, 1, 4, 2, 2, 3, 1, 4] 7
[2, 1, 4, 2, 2, 3, 2, 4] 8
[3, 1, 3, 1, 2, 3, 1, 4] 6
[3, 1, 4, 1, 2, 3, 1, 4] 3
[3, 2, 4, 1, 2, 2, 1, 4] 3
[3, 2, 4, 1, 2, 3, 1, 3] 3
[3, 2, 4, 1, 2, 2, 1, 4] 3
[4, 2, 4, 1, 2, 2, 1, 4] 6
[3, 3, 4, 1, 2, 3, 1, 4] 3
[3, 3, 4, 1, 2, 3, 1, 3] 4
[2, 2, 4, 1, 2, 2, 2, 4] 7
[3, 2, 4, 1, 2, 3, 1, 4] 0
>
```

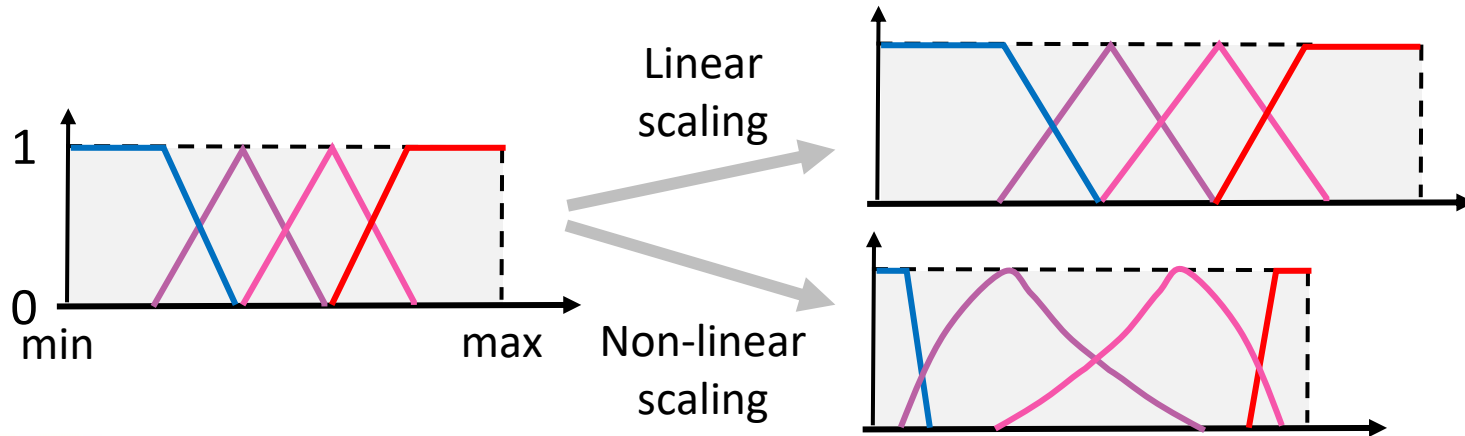

Genetic algorithms for fuzzy systems

- Designing the fuzzy rules and membership functions for a large fuzzy system manually can be a laborious task
 - Genetic algorithms could be used for learning the fuzzy knowledge base, including the fuzzy rules and the membership functions



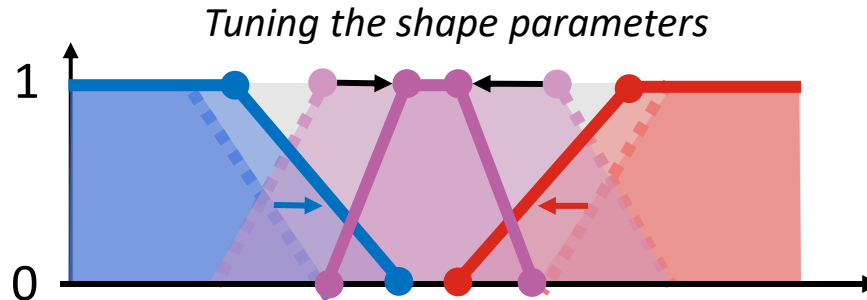
Genetic tuning of membership functions

- In the simplest case, GA is only used to tune membership functions
 - Pre-defined fuzzy rules not changed
 - Linear or non-linear scaling factors or functions can be determined by GA to adapt the fuzzy system to different contexts



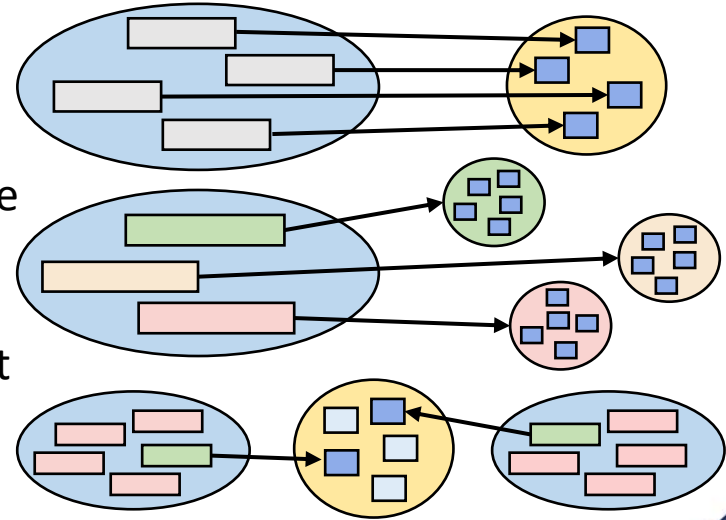
Tuning of membership function shape

- The shape of the membership function can be parametrised and encoded as a chromosome for tuning
 - Most common shapes are triangles and trapezoids that can be represented by their end points and support points
 - Gaussian and sigmoid functions are also parametrised shapes



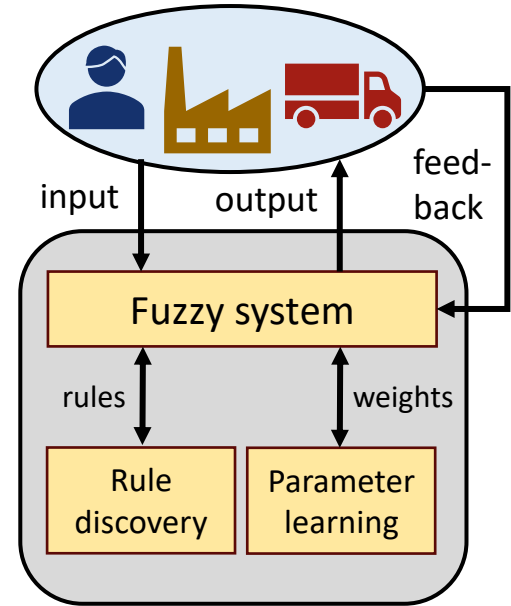
GA approaches to learn fuzzy rules

- Since fuzzy systems vary enormously in size and structure, there are also different approaches to use GAs to design fuzzy systems
 - *Michigan approach*: the whole population represents the rule base, one chromosome is one rule
 - *Pittsburgh approach*: each chromosome encodes a whole rule base
 - *Iterative rule learning (IRL)*: each chromosome represents a one rule, but only the best individual is incorporated in the final rule base



Michigan approach

- The chromosomes represent individual rules
 - Genetic operations applied at the level of an individual chromosome (i.e., no crossovers used)
 - Each chromosome evolves towards the optimum
- Used in *learning classifier systems (LCS)* including discovery and learning parts
 - *Discovery part* uses GA for finding new rules
 - *Learning part* uses other techniques for learning the relative weights between the rules

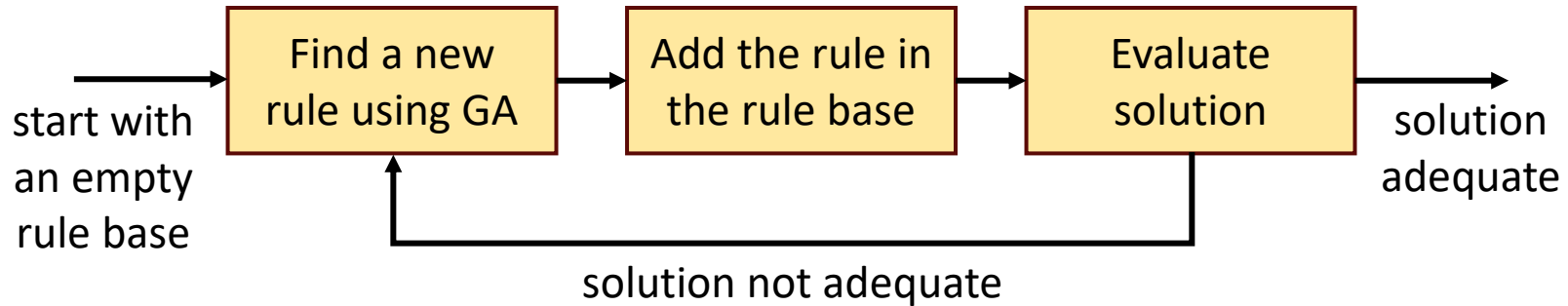


Pittsburgh approach

- Each chromosome encodes a whole fuzzy rule base
 - Crossovers generate new combinations of rules
 - Mutations generate new rules
- Pittsburgh approach improve collaboration between the rules
 - In the Michigan approach rules evolve independently, potentially making the rules to compete against each other
 - In the Pittsburgh approach rules evolve jointly, and therefore they learn naturally to collaborate
- The disadvantage of Pittsburgh approach is the large search space, making it slow and complex to find optimal rule combinations

Iterative rule learning (IRL) approach

- The chromosomes represent individual rules, but only the best rule is adopted in the final rule base
 - Finding one rule is efficient, but the approach requires several iterations of the algorithm to create a full rule base



Summary

- Genetic algorithms (GA) are inspired by biological evolution
 - A set of potential solutions to the problem (chromosomes) are created
 - Fitness function is used to evaluate the suitability of the solution
 - The fittest chromosomes are used to produce offsprings via genetic operators like crossovers and mutations
- Application of GA for designing fuzzy systems presented
 - GAs can be used for tuning the membership functions for a system with predefined fuzzy rules, or for creating the rule base
 - Michigan, Pittsburgh, and iterative rule learning approaches for learning the fuzzy rule base introduced briefly

Questions and discussion